



PICTURE

(PinkOS Interface Construction Terminology for UI Representation & Events)

DISEÑO E IMPLEMENTACIÓN

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Augusto Andrés	Lanari	64679	alanari@itba.edu.ar
Jerónimo	Lopez Vila	65778	jlopezvila@itba.edu.ar
Luca	Mancuso	64150	lmancuso@itba.edu.ar
Santiago	Torres	64563	storres@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en: [PICTURE](#).

3. Dominio

PinkOS es un sistema operativo experimental desarrollado en el marco de las materias Arquitectura de Computadoras y Sistemas Operativos.

Para facilitar el desarrollo de interfaces gráficas en PinkOS, se propone la creación de un lenguaje declarativo llamado PICTURE (PinkOS Interface Construction Terminology for UI Representation & Events).

El objetivo de PICTURE es permitir que los desarrolladores definan interfaces gráficas mediante descripciones textuales, de forma más sencilla que escribir la lógica en C.

Un compilador/transpilador convierte estas descripciones a código C que utiliza las primitivas de PinkOS para renderizar los elementos y manejar eventos.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Toda la interfaz se define como un **árbol jerárquico** de componentes, donde uno se define dentro del otro usando la tabulación.
- (II). Cada componente requiere un **identificador único** dentro de su ventana, definido con #, el cual permitirá referenciarlo desde la lógica en C.
- (III). Cada componente tiene **propiedades** que definen características de estilo, posicionamiento, alineamiento, etc...
- (IV). El **valor de las propiedades** de los componentes puede ser de **tipos de dato variados**, incluyendo algunos built-ins como "s", "m" y "l" para tamaño de texto o "blue", "red", etc... para colores.
- (V). Se pueden definir **variables globales** para usar como valor en las propiedades de los componentes, modificables en el código C.
- (VI). Las propiedades pueden modificarse desde la lógica en C, haciendo posible **UIs dinámicas**.
- (VII). El **orden de render** (z-index) se deriva de la posición en el árbol.
- (VIII). Los componentes pueden definir **callbacks** que determinan qué lógica se asociará a acciones dadas (on_press, on_select y on_keypress).
- (IX). Los **componentes seleccionables** (o sea que hayan definido algún callback que dependa de tener el foco) se iterarán con tab en el orden en que aparezcan en el árbol. En el futuro podría utilizarse una navegación más compleja basada en la posición x,y.
- (X). Hacer **enter** ejecutará el **callback** del componente en foco.
- (XI). Se permitirá escribir comentarios con //

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Una ventana con un texto fijo de un color y tamaño específicos.
- (II). Un botón con un callback `on_press` que cambie el valor de un texto.
- (III). Un panel que divida la pantalla en dos partes iguales.
- (IV). Un componente con varios hijos todos utilizando la misma variable; todas deberían modificarse a la par cuando se modifique la variable en la lógica.
- (V). Un comentario.
- (VI). Un input de texto que permita ver lo que el usuario ingresa.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un ID duplicado.
- (II). Una propiedad inexistente.
- (III). Un tipo de dato incorrecto en una propiedad.
- (IV). Una variable utilizada en dos propiedades con tipos de dato distintos.
- (V). Tabulación incorrecta en las propiedades de un componente.
- (VI). Tabulación incorrecta en el id de un componente.
- (VII). Nombre de callback no compatible con las funciones de C.

6. Ejemplos

Ejemplo sencillo con un comentario para visibilizar el formato.

```
---  
- textSize: m  
- color: red  
---  
  
// Esto es un comentario  
# background  
  - background: red  
  - align: center  
# button1  
  - text: "Hola Mundo!"  
  - font_size: textSize  
  - on_press: func  
  - background: color
```

Un “Hola Mundo” con botón que cambia color y usa variables globales.

```
---
- textSize: m
- color: blue
---

# main
- background: white

# background
- background: color
- width: 100
- height: 100

# title
- text: “Hola Mundo!”
- font_size: textSize
- x: 20
- y: 20

# button1
- text: “Cambiar color”
- x: 20
- y: 70
- on_press: handle_press_color
```